UNIVERSIDAD TÉCNICA NACIONAL



INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

ITI-922 SEGURIDAD DE TI II

FUNCIONES CRIPTOGRÁFICAS: HASHES Y CONTRASEÑAS

ANYELINA CHACÓN MORA

IVANNIA PORRAS MIRANDA

FIORELLA ZAMORA CAMPOS

HAROLD SABORÍO ARCE

21/10/2026

Introducción

Casi todo sistema guarda contraseñas, y casi nunca debería guardarlas tal cual. Si una base de datos se filtra, las claves en texto plano quedan expuestas al instante. El tema de este proyecto es cómo almacenar contraseñas para que, incluso tras un robo de la base, sigan siendo difíciles de recuperar.

La herramienta central es la función hash criptográfica: convierte la contraseña en una huella de una sola vía, de la que no se puede volver a la entrada original. Pero el problema no se resuelve solo con elegir un algoritmo "fuerte". Una función hash pensada para verificar integridad de archivos rápidos a propósito es justo lo que un atacante necesita para probar miles de millones de claves por segundo.

Por eso guardar contraseñas exige algo distinto: funciones lentas a propósito, con sal para que dos claves iguales no produzcan el mismo hash, y en los casos modernos con dureza de memoria para encarecer el ataque con hardware especializado. Este informe investiga esas propiedades, las demuestra con una práctica construida por el grupo (un sistema de login que guarda cada contraseña con cuatro esquemas a la vez: MD5, SHA256, bcrypt y Argon2id) y analiza qué pasa cuando el mecanismo se usa mal.

La pregunta que el informe responde es: **¿qué protege este mecanismo, cómo lo demostramos y cómo se rompe si se usa mal?**

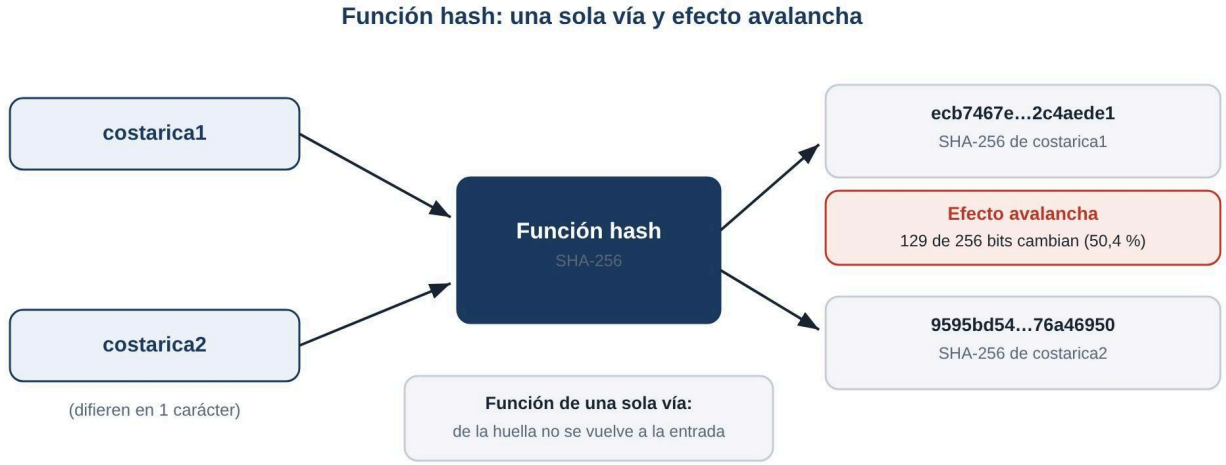
Propiedades y familia de algoritmos

Una función hash criptográfica toma una entrada de cualquier tamaño y devuelve una huella de tamaño fijo. Para ser útil en seguridad debe cumplir tres propiedades de resistencia.

- **Resistencia a preimagen (una sola vía):** dado un hash h, debe ser inviable encontrar una entrada m tal que hash(m) = h. Es lo que impide "devolver" el hash a la contraseña original.
- **Resistencia a segunda preimagen:** dada una entrada m_1, debe ser inviable encontrar otra m_2 distinta con el mismo hash. Protege la integridad de un archivo concreto.
- **Resistencia a colisiones:** debe ser inviable encontrar dos entradas cualesquiera $m_1 \neq m_2$ con el mismo hash. Es la más difícil de sostener y la primera que cae cuando un algoritmo "se rompe".

Efecto avalancha

Un cambio mínimo en la entrada debe cambiar aproximadamente la mitad de los bits de la salida. Lo comprobamos en la práctica: costarica1 y costarica2 difieren en un solo carácter, pero sus hashes SHA-256 difieren en 129 de 256 bits (50,4 %). Gracias a esto, el hash no conserva ninguna pista de la entrada y no se puede "ir acercando" al valor objetivo adivinando.



Dos entradas casi idénticas producen huellas completamente distintas; de la huella no se puede volver a la entrada.

Familia de algoritmos y cuáles están rotos

No todas las funciones hash siguen siendo seguras. MD5 y SHA-1 ya tienen colisiones prácticas demostradas y no deben usarse para firmas ni integridad con valor de seguridad.

Algoritmo	Salida	Estado	Detalle
MD5	128 bits	Roto	Colisiones prácticas desde 2004 a 2008; se generan dos archivos distintos con el mismo hash en segundos
SHA-1	160 bits	Roto	Ataque *SHAttered* (Google/CWI, 2017): primera colisión real; en 2020 se logró colisión con prefijo elegido
SHA-2 (SHA256/512)	256 / 512 bits	Vigente	Sin colisiones conocidas; es el estándar actual para integridad y firmas
SHA-3 (Keccak)	224–512 bits	Vigente	Diseño distinto (construcción *sponge*), estandarizado por NIST en 2015 como alternativa a SHA-2

Estar "roto" por colisiones no es lo único que importa para contraseñas. Como se verá en la sección 4, incluso un algoritmo íntegro como SHA-256 es inadecuado para guardar contraseñas, porque es demasiado rápido.

Usos de las funciones hash y HMAC

Las funciones hash aparecen en muchos mecanismos de seguridad, no solo en contraseñas:

- **Integridad de archivos:** se publica el hash de una descarga; si el archivo se altera, su hash cambia
- **Firmas digitales:** se firma el hash del documento, no el documento entero, porque es más corto y de tamaño fijo
- **HMAC:** código de autenticación de mensajes basado en hash, cada bloque encadena el hash del
- bloque anterior, así cualquier cambio rompe la cadena.

Almacenamiento de contraseñas: el tema central de este proyecto (secciones 4 a 6).

HMAC: qué problema resuelve

Un hash simple solo da integridad: si el mensaje cambia, el hash cambia. El problema es que cualquiera puede recalcular el hash del mensaje alterado, así que un hash suelto no prueba quién generó el mensaje.

HMAC añade una llave secreta compartida entre emisor y receptor. Solo quien tiene la llave puede generar o verificar el código. Si el mensaje se altera, la verificación falla; y sin la llave, un atacante no puede fabricar un HMAC válido. Así, HMAC da integridad y autenticidad, no solo integridad.



El emisor envía el mensaje junto con su HMAC; el receptor lo recalcula con la misma llave y compara

Derivación de claves (KDF): sal, costo ydureza de memoria

Guardar contraseñas con un hash rápido (MD5, SHA-256) es un error, aunque el algoritmo sea íntegro: su velocidad es justo lo que necesita un atacante para probar miles de millones de claves por segundo.

Para esto existen las funciones de derivación de claves (KDF), diseñadas para ser lentas a propósito. Sus tres ideas clave son:

- **Sal (*salt*):** un valor aleatorio distinto por usuario que se mezcla con la contraseña antes de hashear. Hace que dos usuarios con la misma clave tengan hashes distintos y anula las tablas precalculadas (sección 5).
- **Costo (*work factor*):** un parámetro que multiplica el trabajo de calcular cada hash. En bcrypt es el número de rondas (rounds=12 → 2^{12} = 4096 iteraciones); se sube con el tiempo a medida que el hardware mejora.
- **Dureza de memoria (*memory-hard*):** exigir mucha RAM por cada cálculo, para encarecer los ataques con GPU o hardware especializado, que tienen mucho poder de cómputo pero memoria limitada. Es la ventaja de Argon2.

Las cuatro KDF más usadas

KDF	Año	Sal	Costo	Dureza de memoria
PBKDF2	2000	Sí	Iteraciones	No (vulnerable a GPU)
bcrypt	1999	Sí	Factor de costo	Parcial (4 KB fijos)
scrypt	2009	Sí	Tiempo + memoria	Sí (configurable)
Argon2	2015	Sí	Tiempo + memoria + paralelismo	Sí (el más fuerte)

Argon2 ganó la *Password Hashing Competition* (2015) y tiene tres variantes: Argon2d, Argon2i y Argon2id (híbrida, la recomendada). En nuestra práctica usamos Argon2id con los parámetros mínimos de OWASP: 19 MiB de memoria, 2 iteraciones y paralelismo 1; y bcrypt con factor de costo 12.

Ataques a contraseñas y el papel de la sal

Cuando un atacante roba la base de datos, no ve las contraseñas, solo los hashes. Para recuperarlas prueba candidatos y compara. Los ataques principales son:

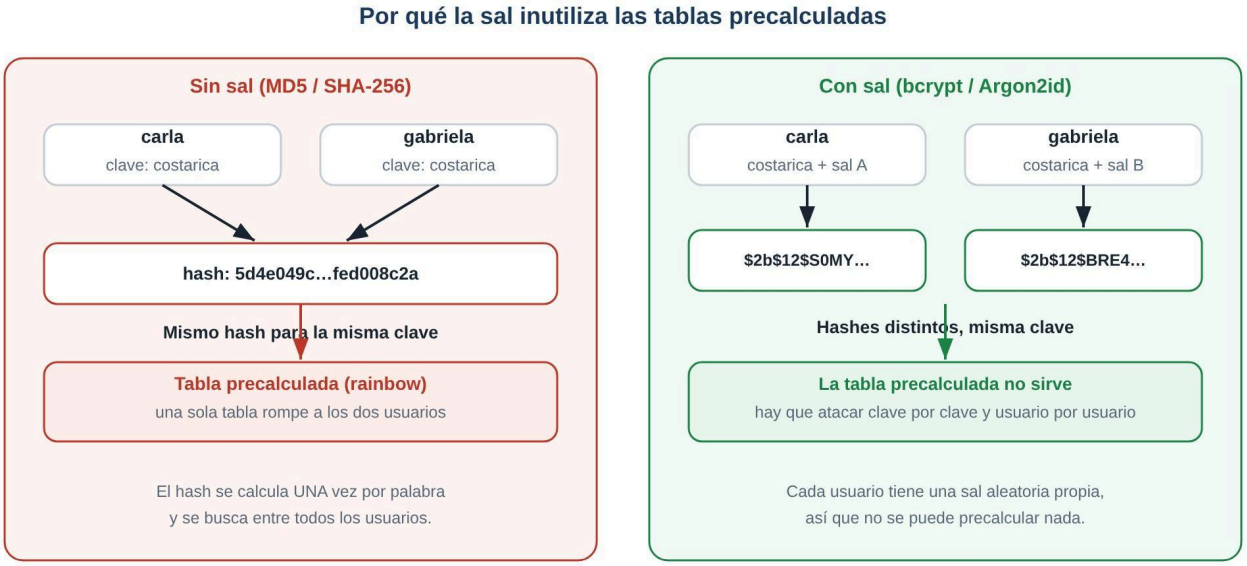
- **Fuerza bruta:** probar todas las combinaciones posibles. Inviable para claves largas y aleatorias, pero rápido contra claves cortas.

  Diccionario: probar una lista de contraseñas comunes y filtradas (123456 , password , qwerty). Es el ataque más rentable, porque la gente reutiliza claves débiles.

Tablas arcoíris (*rainbow tables*): tablas precalculadas que relacionan hash con contraseña. Permiten "buscar" el hash en vez de recalcularlo, a cambio de mucho espacio en disco.

Cómo la sal inutiliza las tablas precalculadas

Las tablas arcoíris y el precalculo solo funcionan si la misma contraseña produce siempre el mismo hash. La sal rompe justamente eso: al mezclar una sal aleatoria distinta por usuario, dos personas con la clave costarica obtienen hashes diferentes, y la tabla precalculada deja de servir. El atacante se ve obligado a atacar clave por clave y usuario por usuario, lo que combinado con una KDF lenta vuelve el ataque carísimo.



Sin sal (MD5/SHA-256), una sola tabla rompe a los dos usuarios; con sal (bcrypt/Argon2id), la misma clave da hashes distintos y no se puede precalcular.

Recomendaciones actuales

Las guías de referencia coinciden en no usar hashes rápidos para contraseñas y en preferir KDF lentas con sal. La OWASP Password Storage Cheat Sheet recomienda, en orden:

1. **Argon2id** como primera opción, con un mínimo de 19 MiB de memoria, 2 iteraciones y paralelismo 1
2. **scrypt** si Argon2 no está disponible
3. **bcrypt** para sistemas que ya lo usan, con un factor de costo de al menos 10 (usamos 12).
4. **PBKDF2** solo cuando se exige cumplimiento FIPS, con muchas iteraciones.

Complementos recomendados: añadir un *pepper* (un secreto del servidor, guardado aparte de la base de datos) y migrar los hashes antiguos a un esquema fuerte cuando el usuario inicie sesión.

El NIST SP 800-63B va en la misma línea: exige almacenar las contraseñas con una función de derivación con sal y costo, y recomienda verificar las nuevas contraseñas contra listas de claves comprometidas en vez de obligar a cambios periódicos o reglas de composición complejas.

Caso real La filtración de LinkedIn (2012)

En junio de 2012, LinkedIn sufrió una brecha en la que se expusieron los hashes de contraseñas de sus usuarios. El problema no fue solo el robo, sino cómo estaban guardadas: con SHA-1 y sin sal.

Qué pasó:

- Inicialmente se publicaron unos 6,5 millones de hashes SHA-1 en un foro. Al no tener sal, los atacantes aplicaron diccionarios y tablas precalculadas y descifraron millones de claves en pocos días.
- En 2016 se reveló que la brecha era mucho mayor: cerca de 117 millones de pares de correo y contraseña salieron a la venta. LinkedIn tuvo que volver a invalidar contraseñas cuatro años después del incidente original.

Por qué fue tan grave: SHA-1 es una función rápida y, sin sal, dos usuarios con la misma clave tenían el mismo hash. Eso permitió precalcular una sola vez y romper a muchos usuarios a la vez, exactamente el escenario de la Figura 3. Una KDF lenta con sal (bcrypt o Argon2id) habría hecho el ataque órdenes de magnitud más caro.

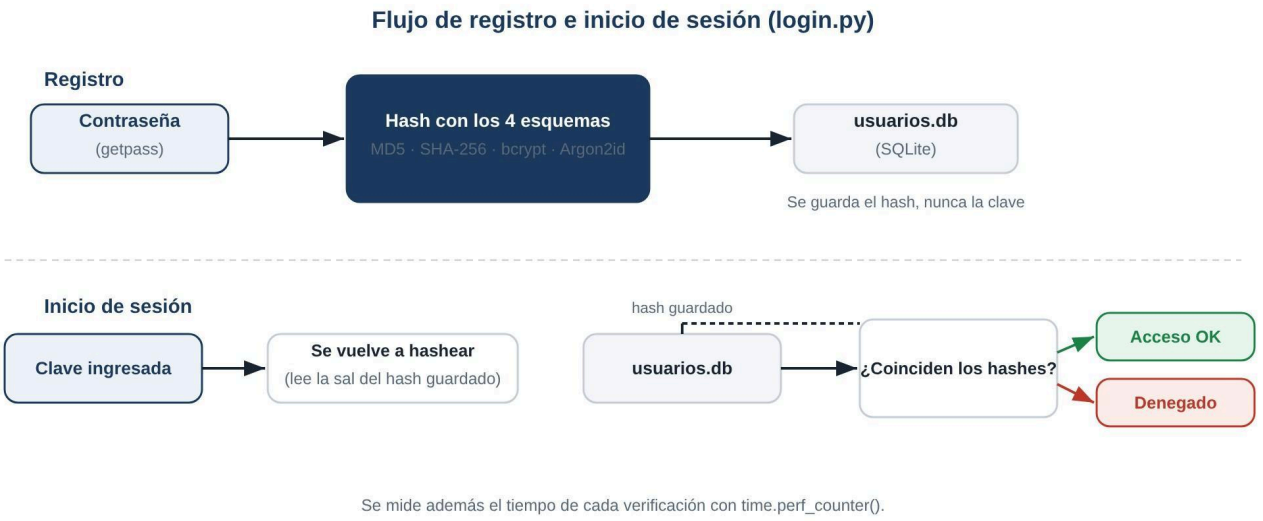
Lección: elegir una función hash "conocida" no basta. El diseño del almacenamiento sal por usuario y costo deliberado es lo que decide si una base filtrada se convierte o no en un desastre. Es la misma conclusión que demuestra nuestra práctica con MD5 y SHA-256 sin sal.

Descripción de la práctica

Es un sistema de registro e inicio de sesión en consola que guarda cada contraseña con los cuatro esquemas a la vez (MD5, SHA-256, bcrypt y Argon2id), más cuatro experimentos que atacan y analizan esos hashes. Todo se ejecutó en la VM Ubuntu del curso. Solo se usaron contraseñas de prueba inventadas por el grupo

Arquitectura

El núcleo es login.py , que expone funciones de hash, verificación y acceso a una base SQLite (usuarios.db). Los demás scripts importan esas funciones sin modificarlas, de modo que atacan y analizan exactamente los mismos hashes que genera el sistema.



En el registro, la contraseña se guarda hasheada con los 4 esquemas; nunca se guarda en texto plano. En el login se vuelve a hashear y se compara.

Archivo	Qué hace
login.py	Registro, login y las funciones de hash/verificación (base del proyecto).
ataque_diccionario.py	Ataque de diccionario contra los 4 esquemas.
experimento_sal.py	Compara los hashes de dos usuarios con la misma clave.
efecto_avalancha.py	Cuenta cuántos bits cambian al variar un carácter.
demo_hmac.py	Genera y verifica un HMAC, intacto y alterado.
graficos.py	Mide tiempos reales y genera los dos gráficos.

Herramientas

Python 3.12 con hashlib , hmac , secrets y sqlite3 (incluidos); bcrypt==5.0.0 y argon2-cffi==25.1.0 para las KDF; John the Ripper (versión *jumbo*) para validar el formato de los hashes exportados; y matplotlib para los gráficos. Parámetros de seguridad: bcrypt con rounds=12 y Argon2id con 19 MiB / 2 iteraciones / paralelismo 1 (OWASP).

Resultado 1 Ataque de diccionario

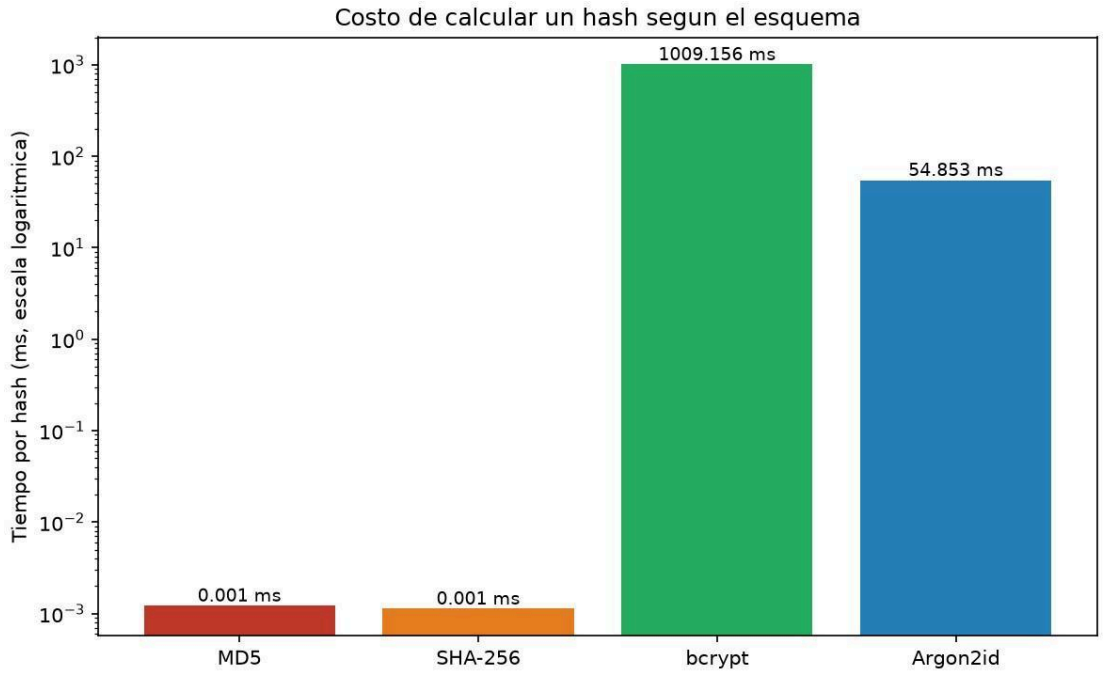
Con un diccionario de 24 palabras y 9 usuarios de prueba, el ataque mostró una diferencia enorme entre los esquemas sin sal y los esquemas con sal:

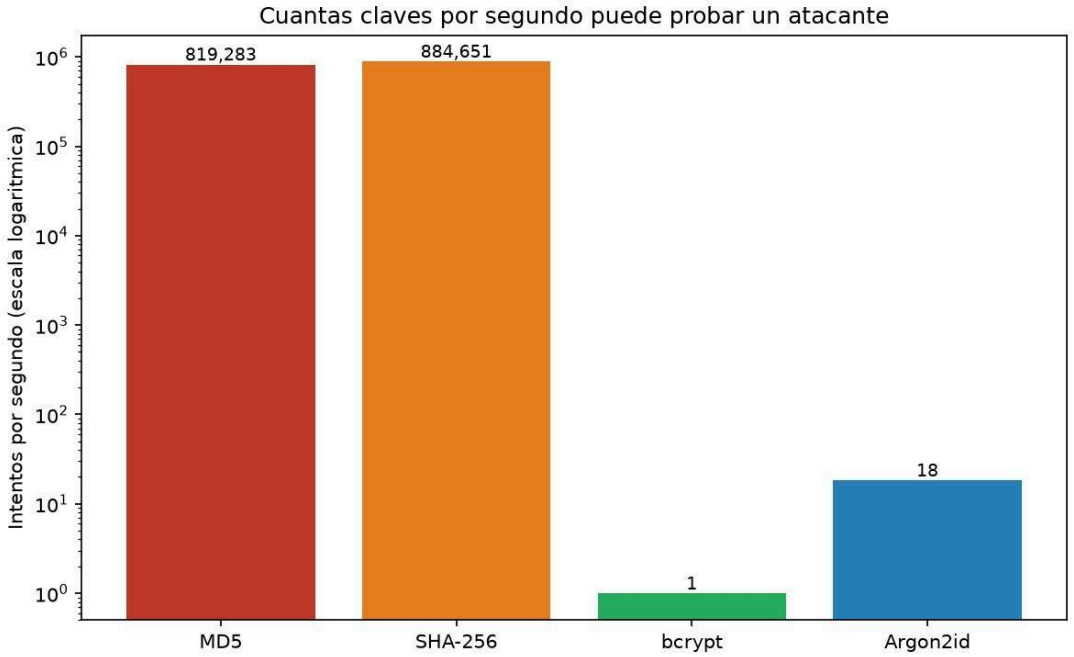
Esquema	Velocidad del ataque	Usuarios rotos
MD5	240 906 intentos/seg	6 de 9
SHA-256	437 693 intentos/seg	6 de 9
bcrypt	1,6 intentos/seg	3 de 4 (muestra)
Argon2id	17,8 intentos/seg	3 de 4 (muestra)

Atacar bcrypt resultó unas 154 000 veces más lento que atacar MD5. Las claves débiles (123456 , password , qwerty , costarica) cayeron en todos los esquemas, pero en MD5/SHA-256 al instante y en bcrypt/Argon2id a un costo altísimo. La clave aleatoria de 16 caracteres no cayó en ningún esquema: la fortaleza de la clave también importa.

Resultado 2 Costo de calcular un hash

Los gráficos miden por qué la velocidad es una defensa. MD5 y SHA-256 tardan 0,001 ms por hash; Argon2id 55 ms; bcrypt 1009 ms. Esa lentitud es justo lo que encarece el ataque.





A la izquierda, tiempo por hash (escala logarítmica); a la derecha, intentos por segundo que permite cada esquema. MD5 y SHA-256 dejan probar cientos de miles de claves por segundo; bcrypt, apenas una.

Resultado 3 Sal, avalancha e integridad

- **Sal:** carla y gabriela tienen la misma clave (costarica). Su MD5 y su SHA-256 son idénticos, pero su bcrypt y su Argon2id son distintos, porque cada una recibe una sal aleatoria
- **Efecto avalancha:** cambiar un carácter cambió 50 % de los bits del hash
- **HMAC:** con el mensaje intacto la verificación es válida; al alterar un carácter, falla

Análisis

Los números confirman la teoría: la velocidad de un hash es una ventaja para el atacante, no para la seguridad. MD5 y SHA-256, al ser rápidos y sin sal, permitieron romper todas las claves débiles al instante y con una sola tabla precalculada. bcrypt y Argon2id, lentos y con sal, encarecieron el ataque unas 154 000 veces. La sal evitó que dos claves iguales se delataran, el efecto avalancha mostró la naturaleza de una sola vía y el HMAC añadió autenticidad al control de integridad.

Limitaciones. La prueba se hizo con un diccionario pequeño (24 palabras) y 9 usuarios, sobre una sola máquina virtual; las cifras de intentos por segundo dependen del CPU y varían entre corridas. Para

bcrypt y Argon2id solo se atacó una muestra de usuarios, porque atacar a todos habría tardado demasiado lo cual es, en sí mismo, parte del resultado. No se probó con rockyou.txt completo ni con hashcat sobre GPU, que es el escenario real de un atacante con recursos.

Qué haríamos distinto en un entorno real.

- Guardar **s**olo Argon2id (o bcrypt como alternativa), nunca MD5 ni SHA-256; aquí se guardaron los cuatro únicamente para poder compararlos.
- Añadir un *pepper* (un secreto del servidor fuera de la base de datos) para que, si solo se filtra la base, los hashes sigan protegidos algo que la sal por sí sola no cubre.
- Verificar las contraseñas nuevas contra listas de claves ya filtradas (recomendación de NIST) y aplicar límites de intentos y segundo factor.
- Subir los parámetros de costo periódicamente y migrar los hashes antiguos cuando el usuario inicie sesión.

Conclusiones y recomendaciones

Ivannia Porras. La práctica me dejó claro que el problema no es el algoritmo "fuerte", sino cómo se guarda la contraseña. Ver que bcrypt resultó unas 154 000 veces más lento de atacar que MD5 demuestra que la lentitud es una defensa buscada a propósito. Recomiendo que todo sistema nuevo use Argon2id con sal y descarte MD5 y SHA-256 para contraseñas

Angelina. Lo que más me marcó fue el experimento de la sal: carla y gabriela tenían el mismo MD5 por usar la misma clave, y con bcrypt su hash fue distinto. Entendí por qué la sal inutiliza las tablas precalculadas. Mi recomendación es que la sal sea siempre aleatoria y única por usuario, nunca un valor fijo

Fiorella Zamora Campos. Al construir login.py confirmé que detalles como leer la contraseña con getpass , comparar en tiempo constante y validar el límite de 72 bytes de bcrypt son parte de la seguridad, no adornos. También verificamos los parámetros de Argon2id contra OWASP en vez de asumirlos. Recomiendo seguir guías oficiales y probar cada librería en la propia máquina.

Harold. El caso de LinkedIn 2012 conecta la teoría con la realidad: SHA-1 sin sal convirtió una brecha en la exposición de millones de contraseñas. Una empresa hoy debería guardar contraseñas con Argon2id (o bcrypt), añadir un *pepper*, limitar intentos y revisar las claves contra listas filtradas

Recomendación final del grupo. Para guardar contraseñas hoy: Argon2id con sal única por usuario y costo ajustado (19 MiB / 2 / 1 como mínimo), bcrypt (factor ≥ 12) como alternativa, nunca MD5 ni SHA-256; y como defensas complementarias, *pepper*, límite de intentos y verificación contra contraseñas ya comprometidas.

Referencias

Biryukov, A., Dinu, D., & Khovratovich, D. (2021). *Argon2 memory-hard function for password hashing and proof-of-work applications* (RFC 9106). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9106.html

Grassi, P. A., Garcia, M. E., & Fenton, J. L. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). National Institute of Standards and Technology. https://pages.nist.gov/800-63-3/sp800-63b.html

Krebs, B. (2016, 18 de mayo). *As scope of 2012 breach expands, LinkedIn to again reset passwords for some users*. Krebs on Security. https://krebsonsecurity.com/2016/05/as-scope-of-2012-breach-expands-linkedin-to-again-reset-passwords-for-some-users/

Lomas, N. (2016, 18 de mayo). *117 million LinkedIn emails and passwords from a 2012 hack just got posted online*. TechCrunch. https://techcrunch.com/2016/05/18/117-million-linkedin-emails-and-passwords-from-a-2012-hack-just-got-posted-online/

National Institute of Standards and Technology. (2015). *SHA-3 standard: Permutation-based hash and extendable-output functions* (FIPS PUB 202). https://doi.org/10.6028/NIST.FIPS.202

OWASP Foundation. (2024). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Stevens, M., Bursztein, E., Karpman, P., Albertini, A., & Markov, Y. (2017). *The first collision for full SHA-1* [SHAttered]. https://shattered.io/

Bitácora de uso de IA

El grupo usó IA (Claude Code) para programar, depurar y redactar. Todo el código se probó y verificó en la VM antes de cada commit, y las fuentes externas se consultaron en vez de asumirlas.

Fecha	Herramienta	Qué le pedimos	Qué respondió	Qué verificamos o corregimos
14/10/26	Claude Code	Montar la VM e instalar el entorno.	Instaló dependencias, creó el venv y generó requirements.txt y .gitignore.	Versiones: bcrypt 5.0.0 y argon2-cffi 25.1.0.
14/10/26	Claude Code	Verificar las APIs de bcrypt y Argon2.	Confirmó gensalt(rounds=12) y los parámetros de PasswordHasher.	Comprobamos las firmas con inspect.signature en la VM.
14/10/26	Claude Code	Revisar parámetros Argon2id según OWASP.	Indicó m=19456 KiB, t=2 y p=1; descartó los 15 MiB del borrador.	Consultamos OWASP y usamos 19456/2/1.
15/10/26	Claude Code	Probar bcrypt con claves mayores de 72 bytes.	bcrypt 5.0.0 genera ValueError; no recorta la clave.	Probamos 73 bytes y añadimos validar_largo_bcrypt.
15/10/26	Claude Code (Opus)	Escribir login.py.	Creó hash, verificación, SQLite, menú, getpass y medición de tiempos.	Probamos registro, acceso válido e inválido, usuario inexistente y clave larga.
16/10/26	Claude Code	Resolver el error al importar login con python -I.	Explicó que -I excluye la carpeta actual del path.	Ejecutamos sin -I por tratarse de código propio.
19/10/26	Claude Code	Crear experimentos y gráficos.	Generó pruebas de diccionario, sal, avalancha y HMAC, y dos gráficos PNG.	Ejecutamos en la VM y guardamos los resultados en resultados/.

21/10/26		Bases para el informe	Búsqueda de información, ejemplos de casos reales, ayuda con los gráficos	Contrastamos cifras con resultados/, verificamos el caso LinkedIn y ajustamos la redacción.